

Name: Kanika Saini

Program: Btech CSE B16

SAP ID: 590034098

HDOC Day-18

Implementation of SGPA and CGPA Calculation Using Nested Loops & Decision-Making Constructs in C

1. Problem Statement

Write a C program to calculate the **SGPA (Semester Grade Point Average)** for each semester and the final **CGPA (Cumulative Grade Point Average)** for m semesters.

Each semester may contain a different number of courses. The program must validate all inputs and assume that the student has passed all courses.

The program uses:

- **Nested loops** to process semesters and courses.
 - **Decision-making constructs** to validate inputs and convert grades into grade points.
 - **Loops for validation** so that incorrect input is entered again.
-

2. Input, Processing and Output

Inputs

The program accepts:

- `global_id` – Student Global ID
- `m` – Number of semesters
- `n` – Number of courses in each semester
- `code` – Course code

- `credit` – Credits of the course
- `grade` – Grade obtained

All numerical inputs listed above must be positive where applicable, and grades must belong to the allowed set.

Grade Point Conversion

The university conversion scale given in the question is:

Grade Grade Point (GP)

O	10
A	9
B	8
C	7
D	6
E	5
P	4

Formula for SGPA

$SGPA = \text{Semester Total Grade Credits} / \text{Semester Total Credits}$

where

$\text{Semester TGC} = \sum (GP \times \text{Credit})$

The supplied algorithm calculates `semester_TGC`, `semester_TC`, and then divides them to obtain SGPA.

Formula for CGPA

$CGPA = \text{Cumulative Total Grade Credits} / \text{Cumulative Total Credits}$

3. Algorithm

The following is a clean report-ready version of the algorithm provided in your assignment.

Step 1: Start.

Step 2: Declare the required integer, character and floating-point variables.

Step 3: Input `global_id`.

Step 4: Check whether `global_id > 0`. If not, display an error and ask the user to enter it again.

Step 5: Input number of semesters `m`.

Step 6: Check whether `m > 0`. If not, display an error and input it again.

Step 7: Initialize:

```
cumulative_TC = 0  
cumulative_TGC = 0
```

Step 8: Repeat for every semester from 1 to `m`.

Initialize:

```
semester_TC = 0  
semester_TGC = 0
```

Input the number of courses `n`.

If `n <= 0`, display an error and input it again.

For every course:

1. Input course code.
2. If course code is not positive, ask again.
3. Input course credits.
4. If credits are not positive, ask again.
5. Input grade.
6. Accept only O, A, B, C, D, E, P.
7. Convert grade into GP.
8. Calculate:

```
semester_TGC=semester_TGC+(GP×credit)s
```

9. Calculate:

```
semester_TC=semester_TC+credit
```

After all courses of the semester:

```
SGPA=semester_TGC/semester_TC
```

Display semester number and SGPA.

Update:

```
cumulative_TGC = cumulative_TGC + semester_TGC  
cumulative_TC  = cumulative_TC  + semester_TC
```

Step 9: After completing all semesters, calculate:

CGPA=cumulative_TGC/cumulative_TC

Step 10: Display Global ID, number of semesters, cumulative total credits and CGPA.

Step 11: Stop.

4. Test Case Table

The assignment specifically asks for **valid, invalid and boundary inputs**.

Test Case	Type	Input(s)	Expected Output	Actual Output	Result
1	Valid	ID=590034098, m=1, n=3; (101,4,O), (102,3,A), (103,3,B)	SGPA = 9.10, CGPA = 9.10	SGPA = 9.10, CGPA = 9.10	Pass
2	Valid	ID=590034098, m=1, n=2; (101,4,B), (102,4,C)	SGPA = 7.50, CGPA = 7.50	SGPA = 7.50, CGPA = 7.50	Pass
3	Invalid ID	ID=0 → 590034098	Error for ID=0; accept positive ID	Error shown; valid ID accepted	Pass
4	Invalid Semester	m=0 → 1; n=1; (101,4,A)	Error for m=0; SGPA=9.00	Error shown; SGPA=9.00	Pass
5	Invalid Course Count	n=0 → 1; (101,4,B)	Error for n=0; SGPA=8.00	Error shown; SGPA=8.00	Pass
6	Invalid Grade	n=1; (101,4,X→O)	Error for X; O accepted; SGPA=10.00	Error shown; SGPA=10.00	Pass
7	Boundary	ID=1, m=1, n=1; (1,1,P)	SGPA=4.00, CGPA=4.00	SGPA=4.00, CGPA=4.00	Pass
8	Boundary	ID=1, m=1, n=1; (1,1,O)	SGPA=10.00, CGPA=10.00	SGPA=10.00, CGPA=10.00	Pass

5. C Program

This implementation follows the supplied algorithm, including repeated input validation and the specified grade-point conversion.

```
#include <stdio.h>

int main()
{
    int global_id, m, n;
    int semester, course;
    int code, credit, GP;
    int semester_TC, semester_TGC;
    int cumulative_TC = 0, cumulative_TGC = 0;
    char grade;
    float SGPA, CGPA;

    /* Input and validation of Global ID */
    do
    {
        printf("Enter Global ID: ");
        scanf("%d", &global_id);

        if (global_id <= 0)
            printf("Invalid Global ID! Enter a positive value.\n");
    } while (global_id <= 0);

    /* Input and validation of number of semesters */
    do
    {
        printf("Enter number of semesters: ");
        scanf("%d", &m);

        if (m <= 0)
            printf("Invalid number of semesters! Enter again.\n");
    } while (m <= 0);

    /* Loop for semesters */
    for (semester = 1; semester <= m; semester++)
    {
        semester_TC = 0;
        semester_TGC = 0;

        printf("\n--- Semester %d ---\n", semester);

        /* Input and validation of number of courses */
        do
        {
            printf("Enter number of courses: ");
            scanf("%d", &n);

            if (n <= 0)
                printf("Invalid number of courses! Enter again.\n");
        } while (n <= 0);
```

```

/* Loop for courses */
for (course = 1; course <= n; course++)
{
    printf("\nCourse %d\n", course);

    /* Course code validation */
    do
    {
        printf("Enter course code: ");
        scanf("%d", &code);

        if (code <= 0)
            printf("Invalid course code! Enter again.\n");
    } while (code <= 0);

    /* Credit validation */
    do
    {
        printf("Enter course credit: ");
        scanf("%d", &credit);

        if (credit <= 0)
            printf("Invalid credit! Enter again.\n");
    } while (credit <= 0);

    /* Grade validation */
    do
    {
        printf("Enter grade (O/A/B/C/D/E/P): ");
        scanf(" %c", &grade);

        if (grade != 'O' && grade != 'A' &&
            grade != 'B' && grade != 'C' &&
            grade != 'D' && grade != 'E' &&
            grade != 'P')
        {
            printf("Invalid grade! Enter again.\n");
        }
    } while (grade != 'O' && grade != 'A' &&
        grade != 'B' && grade != 'C' &&
        grade != 'D' && grade != 'E' &&
        grade != 'P');

    /* Convert grade into Grade Point */
    switch (grade)
    {
        case 'O':
            GP = 10;
            break;

        case 'A':
            GP = 9;
            break;
    }
}

```

```

        case 'B':
            GP = 8;
            break;

        case 'C':
            GP = 7;
            break;

        case 'D':
            GP = 6;
            break;

        case 'E':
            GP = 5;
            break;

        case 'P':
            GP = 4;
            break;
    }

    /* Calculate semester totals */
    semester_TGC = semester_TGC + (GP * credit);
    semester_TC = semester_TC + credit;
}

/* Calculate SGPA */
SGPA = (float)semester_TGC / semester_TC;

printf("\nSemester %d SGPA = %.2f\n",
        semester, SGPA);

/* Update cumulative values */
cumulative_TGC = cumulative_TGC + semester_TGC;
cumulative_TC = cumulative_TC + semester_TC;
}

/* Calculate CGPA */
CGPA = (float)cumulative_TGC / cumulative_TC;

printf("\n==== FINAL RESULT =====\n");
printf("Global ID      : %d\n", global_id);
printf("Total Semesters  : %d\n", m);
printf("Cumulative Credits : %d\n", cumulative_TC);
printf("CGPA              : %.2f\n", CGPA);

return 0;
}

```

6. Sample Execution — Valid Test Case

For Test Case 1:

Enter Global ID: 590034098
Enter number of semesters: 1

--- Semester 1 ---
Enter number of courses: 3

Course 1
Enter course code: 101
Enter course credit: 4
Enter grade (O/A/B/C/D/E/P): O

Course 2
Enter course code: 102
Enter course credit: 3
Enter grade (O/A/B/C/D/E/P): A

Course 3
Enter course code: 103
Enter course credit: 3
Enter grade (O/A/B/C/D/E/P): B

Semester 1 SGPA = 9.10

===== FINAL RESULT =====
Global ID : 590034098
Total Semesters : 1
Cumulative Credits : 10
CGPA : 9.10

Manual Verification

Course 1:

$$10 \times 4 = 40$$

Course 2:

$$9 \times 3 = 27$$

Course 3:

$$8 \times 3 = 24$$

Therefore:

$$\text{Semester TGC} = 40 + 27 + 24 = 91$$

and

$$\text{Semester TC} = 4 + 3 + 3 = 10$$

Hence:

SGPA=9/10=9.10

Since there is only one semester:

CGPA=9.10

Therefore, **Expected Output = Actual Output → PASS.**

7. Sample Execution — Invalid Input

```
Enter Global ID: 0
Invalid Global ID! Enter a positive value.
Enter Global ID: 590034098

Enter number of semesters: 0
Invalid number of semesters! Enter again.
Enter number of semesters: 1

--- Semester 1 ---
Enter number of courses: 0
Invalid number of courses! Enter again.
Enter number of courses: 1

Course 1
Enter course code: -101
Invalid course code! Enter again.
Enter course code: 101

Enter course credit: 0
Invalid credit! Enter again.
Enter course credit: 4

Enter grade (O/A/B/C/D/E/P): X
Invalid grade! Enter again.
Enter grade (O/A/B/C/D/E/P): A

Semester 1 SGPA = 9.00

===== FINAL RESULT =====
Global ID           : 590034098
Total Semesters     : 1
Cumulative Credits  : 4
CGPA                 : 9.00
```

This demonstrates the repeated validation required by the supplied algorithm.

8. Testing and Evaluation

Test Case 1: Valid inputs produce SGPA 9.10 and CGPA 9.10. **PASS**

Test Case 2: Valid grades B and C with equal credits produce SGPA 7.50. **PASS**

Test Case 3: `global_id = 0` is rejected and the program asks for another value. **PASS**

Test Case 4: `m = 0` is rejected because the number of semesters must be positive. **PASS**

Test Case 5: `n = 0` is rejected because each semester must have a positive number of courses. **PASS**

Test Case 6: Grade x is rejected, while valid grade o is accepted. **PASS**

Test Case 7: The lower valid grade boundary `P = 4` produces SGPA 4.00. **PASS**

Test Case 8: The upper valid grade boundary `o = 10` produces SGPA 10.00. **PASS**

Thus, the program satisfies the specified algorithm for the prepared test cases.

9. Conclusion

The C program successfully calculates the **SGPA for each semester** and the final **CGPA** using **nested loops and decision-making constructs**. It also validates Global ID, semesters, courses, course codes, credits and grades before performing calculations.

The program correctly uses the specified grade-point scale and calculates cumulative credits and grade credits as required by the given algorithm.

All prepared valid, invalid, and boundary test cases pass successfully.